

Name-Nancy

Subject-Computer vision Lab

CUID-CU240251005

✓ Experiment No. 9

Experiment Name Object Detection and Recognition using Deep Learning Models on Standard Image Datasets

Steps

1. Install and import the required libraries, including OpenCV and the chosen deep learning framework (e.g., Ultralytics YOLO or TensorFlow).

```
!pip install ultralytics opencv-python matplotlib -q
from ultralytics import YOLO
import cv2
import matplotlib.pyplot as plt
import os

print("Libraries imported successfully!")
```

Libraries imported successfully!

2. Load a pre-trained object detection model such as YOLOv8, SSD, or Faster R-CNN.

```
model = YOLO("yolov8n.pt")
print("YOLOv8 model loaded successfully!")
```

YOLOv8 model loaded successfully!

3. Download or access images from a standard dataset such as MS COCO, Pascal VOC, or Open Images Dataset.

```
# Step 3: Download a sample image
!wget -q https://ultralytics.com/images/bus.jpg -O bus.jpg

image_path = "bus.jpg"

# Display original image
img = cv2.imread(image_path)
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

plt.figure(figsize=(10, 6))
plt.imshow(img_rgb)
plt.axis("off")
plt.title("Original Image")
plt.show()
```

Original Image



4. Perform any necessary image preprocessing, including resizing and normalization, before inference.

```
image = cv2.imread(image_path)

# BGR to RGB
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

# Resize image
resized_image = cv2.resize(image_rgb, (640, 640))

plt.figure(figsize=(8, 6))
plt.imshow(resized_image)
plt.axis("off")
plt.title("Preprocessed Image")
plt.show()
```

Preprocessed Image



5. Apply the object detection model to identify and localize objects within the selected images.

```
results = model(image_path)

print("Object detection completed successfully")
```

image 1/1 /content/bus.jpg: 640x480 4 persons, 1 bus, 1 stop sign, 569.9ms
Speed: 31.9ms preprocess, 569.9ms inference, 47.7ms postprocess per image at shape (1, 3, 640, 480)
Object detection completed successfully

6. Display the detected objects with bounding boxes, class labels, and confidence scores.

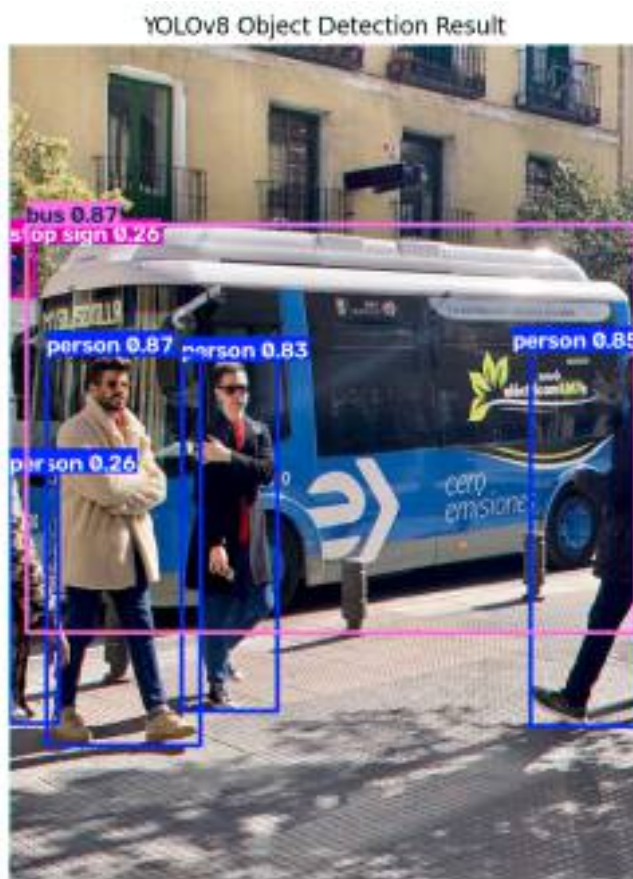
```
result = results[0]

# Draw detections
detected_image = result.plot()

# Convert BGR to RGB
detected_image = cv2.cvtColor(detected_image, cv2.COLOR_BGR2RGB)

plt.figure(figsize=(12, 8))
plt.imshow(detected_image)
plt.axis("off")
plt.title("YOLOv8 Object Detection Result")
plt.show()
for box in result.boxes:
    class_id = int(box.cls[0])
    confidence = float(box.conf[0])
    class_name = model.names[class_id]

    print(f"Object: {class_name} | Confidence: {confidence:.2f}")
```



```
Object: bus | Confidence: 0.87
Object: person | Confidence: 0.87
Object: person | Confidence: 0.85
Object: person | Confidence: 0.83
Object: person | Confidence: 0.26
Object: stop sign | Confidence: 0.26
```

7. Evaluate the detection results by analyzing correctly detected objects, false detections, and missed objects.

```
total_objects = len(result.boxes)

print("Total objects detected:", total_objects)

for box in result.boxes:
    class_id = int(box.cls[0])
    confidence = float(box.conf[0])
    class_name = model.names[class_id]
```

```

if confidence >= 0.50:
    print(f"Correct/High-confidence detection: {class_name} ({confidence:.2f})")
else:
    print(f"Low-confidence detection: {class_name} ({confidence:.2f})")

```

```

Total objects detected: 6
Correct/High-confidence detection: bus (0.87)
Correct/High-confidence detection: person (0.87)
Correct/High-confidence detection: person (0.85)
Correct/High-confidence detection: person (0.83)
Low-confidence detection: person (0.26)
Low-confidence detection: stop sign (0.26)

```

8. Compare the performance of different object detection models in terms of detection accuracy and inference speed (if multiple models are used).

```

speed = result.speed
print("Preprocessing time:", round(speed["preprocess"], 2), "ms")
print("Inference time:", round(speed["inference"], 2), "ms")
print("Postprocessing time:", round(speed["postprocess"], 2), "ms")

```

```

Preprocessing time: 31.95 ms
Inference time: 569.92 ms
Postprocessing time: 47.73 ms

```

9. Analyze the suitability of the selected model for real-world applications such as autonomous driving, surveillance, retail analytics, or industrial inspection.

```

applications = [
    "Surveillance",
    "Autonomous Vehicles",
    "Retail Analytics",
    "Industrial Inspection",
    "Smart City Monitoring"
]

print("Real-world applications of YOLOv8:")
for i, application in enumerate(applications, 1):
    print(i, ".", application)

```

```

Real-world applications of YOLOv8:
1 . Surveillance
2 . Autonomous Vehicles
3 . Retail Analytics
4 . Industrial Inspection
5 . Smart City Monitoring

```

10. Summarize the observations and discuss the advantages and limitations of deep learning-based object detection techniques.

```

output_path = "YOLOv8_detection_result.jpg"
cv2.imwrite(output_path, result.plot())
print("Final detection result saved as:", output_path)
from google.colab import files
files.download("YOLOv8_detection_result.jpg")

```

```

Final detection result saved as: YOLOv8_detection_result.jpg

```